

# DOCUMENTAÇÃO TÉCNICA: TABULEIROS DO MUNDO (TDM)

## 1. Arquitetura Geral do Sistema

O sistema adota uma arquitetura monolítica baseada no padrão MVC (Model-View-Controller) adaptado. A stack tecnológica é composta por Node.js com o framework Express no servidor, SQLite3 para persistência de dados relacional e EJS (Embedded JavaScript) para a camada de visualização, combinando renderização no servidor com manipulações interativas no cliente via HTML5 Canvas e Vanilla JavaScript.

## Árvore de Diretórios do Projeto

Para garantir a integração modular do sistema, a estrutura de arquivos deve seguir rigorosamente a organização abaixo:

- **projeto-tdm/**
  - **server.js**: Ponto de entrada (Bootstrap do Express).
  - **db.js**: Camada de Persistência (SQLite3 e Seeding).
  - **package.json**: Manifesto de dependências do Node.js.
  - **routes/**: Controladores e Roteadores.
    - **auth.js**: Fluxo de usuários (Login, Cadastro, Reset).
    - **catalogo.js**: CRUD de jogos.
    - **tabuleiro.js**: Exibição de metadados dinâmicos.
  - **views/**: Templates de Renderização (Server-Side).
    - **catalogo.ejs**: Interface do catálogo e Painel Administrativo.
    - **tabuleiro.ejs**: Detalhamento de Histórias e Regras.
  - **public/**: Recursos Estáticos (Client-Side).
    - **index.html**: Landing Page.
    - **login.html**: Formas de Acesso.
    - **onca.html**: Aplicação do Jogo da Onça (Canvas).
    - **sobre.html**: Conteúdo Institucional.
    - **css/**: **style.css** e **login.css**.
    - **js/**: **script.js** (Comportamento Global).

## 2. Camada de Persistência (db.js)

A gestão de dados é realizada localmente através do engine SQLite3. O script inicia as tabelas de forma serializada (`db.serialize()`) e executa o Self-Seeding para popular o banco caso esteja vazio.

### 2.1. Modelagem do Banco de Dados

| Tabela     | Coluna      | Tipo      | Restrições                           |
|------------|-------------|-----------|--------------------------------------|
| usuario    | id ▾        | INTEGER ▾ | PRIMARY KEY<br>AUTOINCREMENT         |
|            | nome ▾      | TEXT ▾    | NOT NULL                             |
|            | usuario ▾   | TEXT ▾    | NOT NULL UNIQUE (Chave de Login)     |
|            | email ▾     | TEXT ▾    | UNIQUE                               |
|            | telefone ▾  | TEXT ▾    | -                                    |
|            | senha ▾     | TEXT ▾    | NOT NULL (Salted Hash)               |
| jogos_info | id ▾        | INTEGER ▾ | PRIMARY KEY<br>AUTOINCREMENT         |
|            | titulo ▾    | TEXT ▾    | NOT NULL                             |
|            | origem ▾    | TEXT ▾    | -                                    |
|            | jogadores ▾ | TEXT ▾    | -                                    |
|            | categoria ▾ | TEXT ▾    | estrategia, sorte, familia, infantil |
|            | nota ▾      | REAL ▾    | 0.0 a 5.0                            |

### 2.2. Seed Automatizado e Credenciais Padrão

Caso não existam registros, o sistema injeta um perfil administrativo master:

- **Usuário:** adm
- **E-mail:** adm@tdm.com
- **Senha:** tabuleiros@1234 (Criptografada com Bcrypt, 10 Salt Rounds).
- **Massa de Dados:** Contém 36 jogos históricos iniciais, como Mu Torere, Ntxuva, Adugo e Morabaraba.

### 3. Servidor e Middleware (server.js)

Atua como orquestrador, configurando o motor de visualização EJS e os diretórios de templates.

•

JavaScript

```
// Configuração do motor de visualização para arquivos .ejs
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));
```

- **Middlewares Incorporados:**
  - **bodyParser:** Realiza o parse de payloads via formulários ou JSON.
  - **express.static:** Define o diretório `/public` para acesso livre a ativos estáticos.
  - **express-session:** Gerencia sessões em memória sob a chave `'tdm_secret_key_2025'`, controlando flags como `req.session.isAdmin`.

## 4. Camada de Controle e Rotas (Backend)

### 4.1. Módulo de Autenticação (routes/auth.js)

Utiliza a biblioteca **Bcrypt** para segurança.

- **POST /cadastro:** Valida dados e gera o hash da senha antes da persistência.
- **POST /login:** Compara a senha digitada com o hash do banco. Define privilégios de administrador para o usuário 'adm'.
- **POST /rec\_senha:** Valida a identidade e impede a redefinição para uma senha idêntica à atual.

## 4.2. Módulo de Catálogo (routes/catalogo.js)

- **GET /:** Lista jogos em ordem alfabética, tratando valores nulos com `COALESCE` e limitando o resumo da história a 120 caracteres.
- **POST /adicionar:** Rota restrita para inserção de novos títulos.
- **POST /excluir/:id:** Executa a remoção lógica do registro com base no ID.

## 5. Camada de Visualização e Comportamentos (Front-End)

### 5.1. Comportamentos Globais (public/js/script.js)

- **Menu Responsivo:** Alterna estados do container `.main-nav` para dispositivos móveis.
- **Link Ativo:** Identifica a página atual via `window.location.pathname` e aplica a classe `.active`.
- **Animação por Scroll:** Utiliza a `IntersectionObserver` API para performance, disparando animações em elementos `.reveal` com threshold de 0.15.
- **Efeito Flip de Cards:** Garante que apenas um card exiba o verso descritivo por vez através da classe `.virado`.

### 5.2. Motor de Visualização do Catálogo (views/catalogo.ejs)

- **Filtro Dinâmico:** Intercepta `data-title` e `data-category` para filtragem instantânea no cliente.
- **Painel Administrativo:** Exibe condicionalmente ferramentas de exclusão e o modal de inserção de títulos apenas para o administrador.

### 5.3. Visualização de Detalhes (views/tabuleiro.ejs)

Focada na experiência de leitura, esta interface apresenta o detalhamento individual de cada jogo, incluindo o tratamento de quebras de linha para os campos de história e regras provenientes do banco de dados.